

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning ¹
Student Name	Sadhana J
Register Number	3122247001052
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	15.09.2026
Github Repository	Click Here

1 Aim and Objective

To implement and compare the performance of:

- **Model A:** Single-Layer Perceptron Learning Algorithm (PLA), implemented from scratch.
- **Model B:** Multilayer Perceptron (MLP) with hidden layers and non-linear activations.

Hyperparameters such as activation function, optimizer, learning rate, and hidden-layer size are selected and justified for the MLP through systematic tuning (**GridSearchCV**), and the final PLA and tuned MLP models are compared using standard classification metrics.

Dataset Description

- **Dataset:** English Handwritten Characters Dataset.
- **Samples:** 3,410 images of handwritten characters.
- **Classes:** 62 (digits 0–9, uppercase A–Z, lowercase a–z).
- **Raw representation:** Each image was converted to grayscale, resized to 32×32 pixels, and binarized (pixel intensity $> 127 \Rightarrow 1$, else 0), then flattened into a 1,024-dimensional binary feature vector per image.

Exploratory Data Analysis

The distribution of the 62 character classes was visualized to check for class imbalance before training.



Figure 1: Class distribution of the 62 character labels in the English Handwritten Characters dataset.

Inference: The dataset is close to perfectly balanced, with each of the 62 classes contributing roughly $3410/62 \approx 55$ samples. This uniform distribution means the low accuracy observed for PLA is not an artifact of class imbalance (e.g., a majority class dominating predictions); the poor performance stems from the model’s linear decision boundary being unable to separate visually similar characters, not from skewed training data. It also means weighted precision/recall/F1 are close to their macro-averaged counterparts here, since no class disproportionately influences the weighting.

2 Preprocessing Steps

- Each character image (path given in `english.csv`) was loaded, converted to grayscale ('L' mode), resized to 32×32 , thresholded at pixel intensity 127 to obtain a binary image, and flattened into a 1,024-length feature vector.
- The resulting feature matrix (1,024 pixel features + label) was passed through a common preprocessing pipeline:
 - **Missing values:** Checked and none were found.
 - **Constant-value columns:** Pixel columns with a single unique value across all 3,410 images (e.g. border pixels that are always background) were dropped, reducing the feature count from 1,024 to **640** usable pixel features (final dataset shape: 3410×641 , including the label).
 - **Encoding:** The categorical character `label` column was label-encoded using `LabelEncoder` into 62 integer classes.

- **Feature scaling:** The retained pixel features were standardized using `StandardScaler`.
- **Train–test split:** An 80/20 stratified split (`random_state=42`) was used so that all 62 classes are proportionally represented in both the training and test sets.

3 PLA Implementation and Results

The Perceptron Learning Algorithm was implemented from scratch as a one-vs-rest binary perceptron ensemble:

- For each of the 62 classes, a separate weight vector w and bias b are learned using the step-activation update rule:

$$w_{t+1} = w_t + \eta (y - \hat{y}) x, \quad \hat{y} = \text{sign}(w_t \cdot x + b_t)$$

- **Hyperparameters used:** learning rate $\eta = 0.01$, epochs = 5.
- At prediction time, the class whose weight vector gives the highest raw score ($w \cdot x + b$) is chosen (multi-class decision via one-vs-rest scoring).

Table 1: PLA Performance on the Test Set

Metric	Value
Accuracy	0.0894
Precision (weighted)	0.0988
Recall (weighted)	0.0894
F1-score (weighted)	0.0873
Training Time (s)	10.02
Prediction Time (s)	0.0018
ROC-AUC	Not applicable (no <code>predict_proba</code>)

Result: The from-scratch PLA achieves only about 8.9% test accuracy, only marginally better than random guessing among 62 classes ($\approx 1.6\%$), which is expected since a linear, step-activation model cannot capture the non-linear pixel patterns needed to separate 62 visually similar handwritten character classes.

4 MLP Implementation and Results

An MLP (`sklearn.neural_network.MLPClassifier`) with cross-entropy loss and back-propagation was used as Model B.

Hyperparameter Tuning

A grid search (`GridSearchCV`, 3-fold cross-validation, scoring = accuracy) was performed over:

Table 2: MLP Hyperparameter Grid Searched

Parameter	Values Tried
Hidden layer sizes	(64,), (128,)
Activation function	ReLU, Tanh
Optimizer (solver)	Adam
Initial learning rate	0.001, 0.01

Justification for Chosen Hyperparameters

Table 3: Best MLP Hyperparameters (from Grid Search)

Parameter	Selected Value
Hidden layer sizes	(128,)
Activation function	ReLU
Optimizer	Adam
Initial learning rate	0.01
Best 3-fold CV Accuracy	0.3431
Max iterations (final fit)	400

Justification: ReLU was selected over Tanh because it avoids vanishing gradients and trains faster for this size of network. A single hidden layer of 128 neurons outperformed 64 neurons, indicating the extra capacity helps the model separate 62 classes from 640 binary pixel features. Adam was used as it adapts the learning rate per-parameter and converges faster/more reliably than plain SGD for this kind of tabular-pixel data, and among the two learning rates tried, 0.01 converged to a better cross-validated accuracy (0.343) than 0.001 within the iteration budget.

Table 4: Tuned MLP Performance on the Test Set

Metric	Value
Accuracy	0.3622
Precision (weighted)	0.3874
Recall (weighted)	0.3622
F1-score (weighted)	0.3641
Training Time (s)	5.07
Prediction Time (s)	0.0049
ROC-AUC (weighted, OvR)	0.8966

5 A/B Comparison (PLA vs MLP)

Table 5: PLA vs Tuned MLP – Side-by-Side Comparison

Metric	PLA	MLP (Tuned)
Accuracy	0.0894	0.3622
Precision (weighted)	0.0988	0.3874
Recall (weighted)	0.0894	0.3622
F1-score (weighted)	0.0873	0.3641
Training Time (s)	10.02	5.07
ROC-AUC	N/A	0.8966

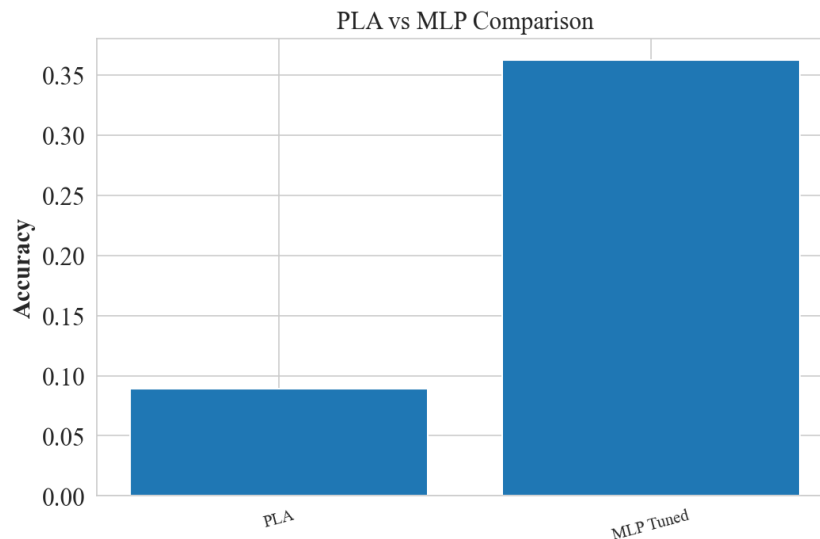


Figure 2: Accuracy comparison between PLA and the tuned MLP.

Inference: The tuned MLP more than quadruples the raw accuracy of the PLA (36.2% vs. 8.9%). This large gap directly reflects the non-linear separability of the 62 handwritten-character classes: PLA can only draw linear decision boundaries between one-vs-rest classes, while the MLP’s hidden layer with ReLU activation allows it to learn curved, composite decision

boundaries which are necessary to distinguish visually similar characters (e.g., ‘O’ vs ‘0’, ‘l’ vs ‘1’).

6 Confusion Matrices and ROC Curves

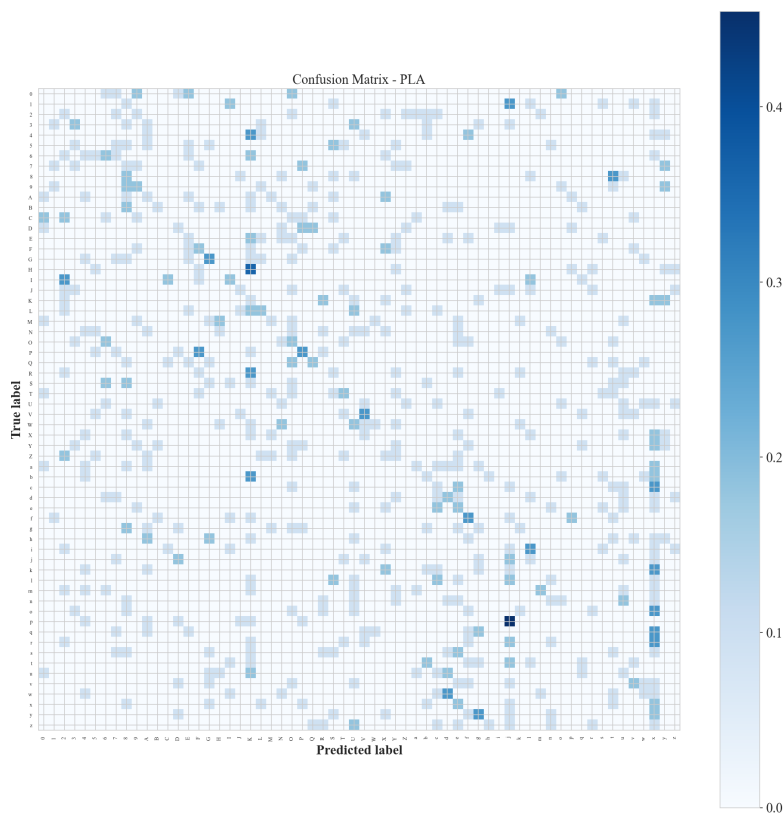


Figure 3: Row-normalized confusion matrix for PLA (62 classes).

Inference: The PLA confusion matrix shows a very diffuse, scattered pattern with no strong diagonal, confirming that predictions are spread thinly and near-randomly across all 62 classes rather than being concentrated on the correct label – consistent with its low accuracy.

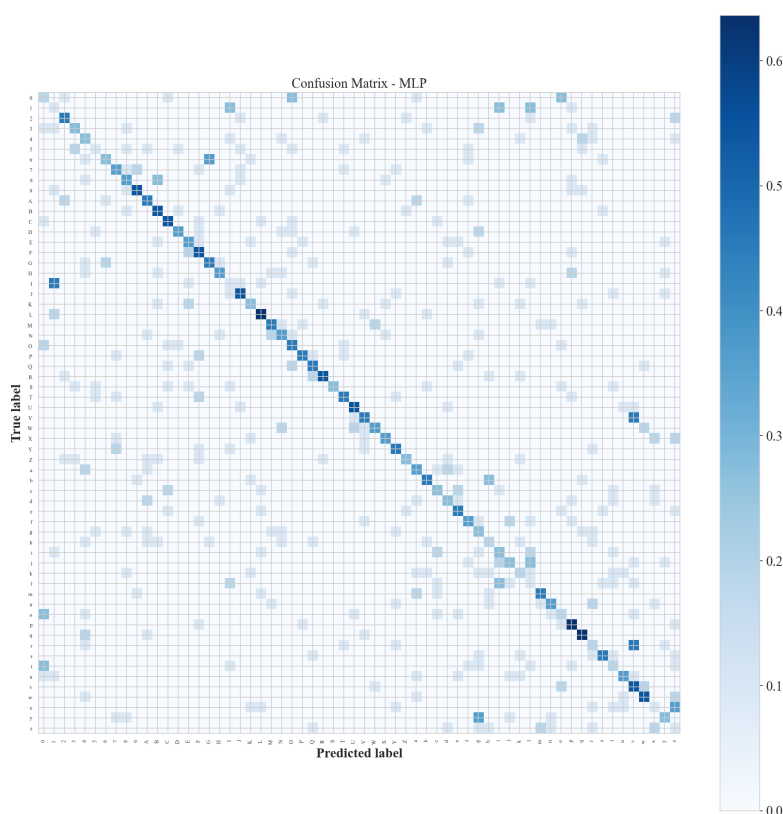


Figure 4: Row-normalized confusion matrix for the tuned MLP (62 classes).

Inference: The MLP confusion matrix shows a visibly stronger diagonal than the PLA's, meaning a much larger share of predictions land on the correct class. Misclassifications that remain are concentrated among characters with very similar binarized shapes (e.g., visually similar uppercase/lowercase or digit/letter look-alikes), which is expected given that only coarse binary pixel features (not grayscale intensity or stroke-based features) were used.

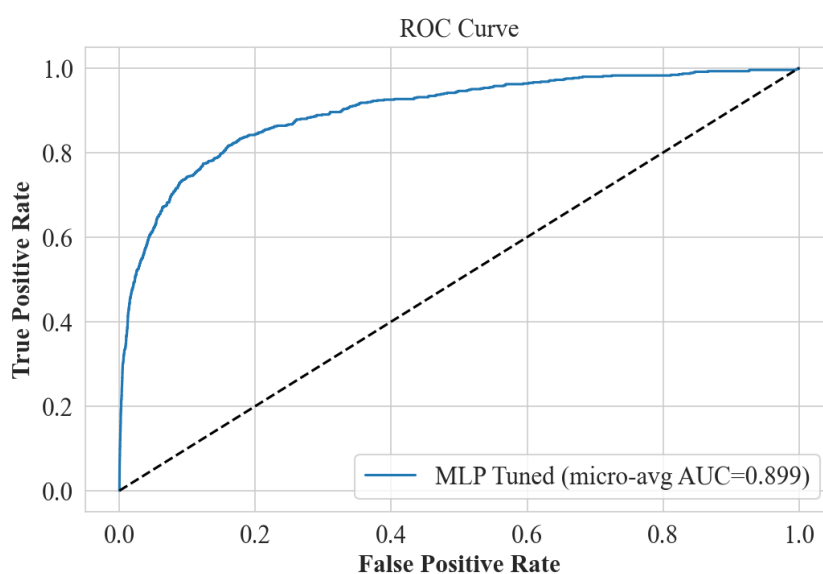


Figure 5: ROC curve (micro-averaged over 62 classes, one-vs-rest) for PLA and the tuned MLP.

Inference: The MLP's ROC curve lies well above the diagonal with a high micro-averaged AUC (weighted OvR ROC-AUC of 0.897 was recorded for the MLP overall), showing strong class-separability in its predicted probabilities. PLA does not expose class probabilities (`predict_proba` is unavailable for the from-scratch implementation), so only its hard predictions could be evaluated, and no ROC-AUC could be computed for it.

7 Conclusion

The tuned Multilayer Perceptron substantially outperforms the from-scratch single-layer Perceptron on the 62-class handwritten character recognition task (36.2% vs. 8.9% test accuracy), confirming the theoretical expectation that a linear, step-activation model cannot handle a task of this non-linear complexity. Grid search identified a single hidden layer of 128 ReLU neurons trained with the Adam optimizer at a learning rate of 0.01 as the best-performing MLP configuration among those tried. While the MLP shows strong class separability (ROC-AUC of 0.897), its moderate absolute accuracy suggests that richer features (e.g., grayscale intensity, HOG, or CNN-based features) and regularization/data augmentation would likely improve results further.